***Abstract*—The ellipse fitting is a fundamental technique used in pattern recognition and computer vision. In this project presents a Vitis HLS-based FPGA accelerator for direct least-squares ellipse fitting applied to shock and vibration data for real-time anomaly detection. The classical Fitzgibbon method formulates the problem as a generalized eigenvalue equation $Sa = \lambda Ca$, where the constraint matrix C is singular. An initial implementation using LDL^T factorization with permutation and Bunch-Kaufman pivoting was explored; however, the complex pivoting logic, multiple matrix inversions, and high resource cost made it unsuitable for efficient hardware realization.**

To address these limitations, a Schur complement reduction was adopted. The 6×6 scatter matrix S is partitioned into blocks, and the reduced 3×3 matrix $\mathbf{S}_{\text{tilde}} = S_{pp} - S_{pq}S_{qq}^{-1}S_{qp}$ is computed. A fixed transformation $A = D^{-1}\mathbf{S}_{\text{tilde}}$ with

$$D^{-1} = \begin{bmatrix} 0 & 0.5 & 0 \\ 0.5 & 0 & 0 \\ 0 & 0 & -1 \end{bmatrix}$$

converts the problem into a standard symmetric 3×3 eigenproblem, which is solved using the cyclic Jacobi method. This approach eliminates LDL^T factorization entirely and is highly synthesizable.

As a result, the C synthesis on a Xilinx Virtex UltraScale+ FPGA yields a latency of 870 cycles (4.35 μs at 200 MHz target) with an initiation interval of 3, using 1432 DSPs, 309k flip-flops, and 310k LUTs. The design maintains numerical agreement with the MATLAB golden reference while enabling real-time ellipse fitting on streaming vibration data.

I. Introduction

The ellipse fitting is one of the most commonly used fittings for pattern recognition and computer vision. However, one of the issues lies in the linear least squares solution, with the conic general equation:

$$F(x, y) = ax^2 + bxy + cy^2 + dx + ey + f = 0 \tag{1}$$

Where the parameters a,b,c,d,e, and f are linear. The polynomial distance F(x,y) is called the algebraic distance of any point (x,y) from the conic. One of the major problems is that the algorithm will most of the time find a way for all the parameters to be zero. For this to be prevented, the parameters must be constrained. The second problem is that it is not guaranteed that the conic will be able to fit an ellipse. For the conic to represent an ellipse, the parameters must satisfy (2);

$$b^2 - 4ac < 0 \tag{2}$$

In the reviewed paper, "Direct Least Squares Fitting to an Ellipse", it demonstrates taking the parameter of vector a, and scaling it to impose the equality constraint:

$$4ac - b^2 = 1 \tag{3}$$

The quadratic constraint can be expressed in the constraint matrix form C (4) to help solve the solution for the generalized eigenvalue problem,

$$C = \begin{bmatrix} 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 0 \\ 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \tag{4}$$

$$Sa = \lambda Ca \tag{5}$$

Where S is the scatter matrix DT * D.

II. Problem Description

The objective of this project is to implement the ellipse fitting onto an FPGA board using Vitis HLS. The purpose will be to compute the ellipse fitting over shock/vibration data generated to detect abnormalities. When first exploring multiple ways to implement the fitting, one of the first methods, the Fitzgibbon method.

This method essentially fits an ellipse to a set of scattered 2D data points(e.g., vibration data). However, this method has proven to be unreliable for hardware use due to how expensive it is to use on hardware. To resolve this, other decomposition methods were considered to replaced the Fitzgibbon method and help solve the general eigenvalue problem, while still being least expensive on the hardware.

The LDL^T factorization is a decomposition method that is closely related to the classical Cholesky factorization. Where L is the lower triangular matrix, D is the diagonal matrix, LT is the upper triangular matrix. This can be written as:

$$S = LDL^T \tag{6}$$

Although the first idea would be to consider using the Cholesky factorization instead of the LDL^T factorization, this has proven to be ineffective when it comes to being used for the ellipse fitting. The Cholesky factorization requires the given symmetric matrix to be positive definite, meaning that all the eigenvalues must be positive. The LDL^T factorization is more flexible with indefinite matrices since it allows the diagonal matrix to have negative or zero entries. In order to use the LDL^T factorization to find the solution for the generalized eigenvalue problem (7), the constraint matrix C must be factorized (8):

$$Sa = \lambda Ca \tag{7}$$

$$C = LDL^T \tag{8}$$

This is done to reduce the generalized eigenvalue problem (7) to a standard symmetric eigenvalue problem, where the LDL^T factorization is used to set C as (9) using the LAPACK Users Guide Third Edition (Generalized Symmetric Definite Eigenproblems).

$$Sa = \lambda LDL^T a \tag{9}$$

III. Proposed Methods

Before the LDL^T factorization can be done, the rows and columns of the C matrix must be reorder through permutation. The LDL^T requires each pivot on the diagonal to be non-zero in order to proceed with an efficient and stable factorization. In the original C matrix, the diagonal entries at positions (1,1)

and (3,3) are equal to zero, while the only usable pivot -1, appears at position (2,2) . If the factorization is attempted like this, it will cause the factorization to fail immediately due to the zero pivot. By applying permutation, the -1 pivot is moved to the leading diagonal position, and the related non-zero elements are grouped into a structure suitable for subsequent pivot steps. Thus, permutation is an important step of the LDL^T factorization for the indefinite matrices such as the constraint matrix.

By applying the permutation method, the only usable pivot in the C matrix (-1) originally at (2,2) is moved to the top-left position (1,1) . Similarly, the off-diagonal entries at (1,3) and (3,1) are relocated so they form a clean 2x2 symmetric block directly below the first pivot. The rearrangement groups the constrained parameters together and puts the non-zero structure of C into a stable pivot pattern: a 1x1 pivot followed by a 2x2 pivot blocks.

$$\text{(C before)} \quad C = \begin{bmatrix} 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 0 \\ 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix};$$

Apply permutation P (swap rows/cols 1 and 2)

$$\text{(C after)} \quad \tilde{C} = PCP^T = \begin{bmatrix} -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix};$$

Fig. 1: Constraint matrix C before and after permutation.

The permutation is also applied to the scatter matrix (S) so that the generalized eigenproblem remains consistent after reordering the variables.

$$S_{3\times3} = \begin{bmatrix} s_{11} & s_{12} & s_{13} \\ s_{12} & s_{22} & s_{23} \\ s_{13} & s_{23} & s_{33} \end{bmatrix} \implies \tilde{S}_{3\times3} = \begin{bmatrix} s_{22} & s_{12} & s_{23} \\ s_{12} & s_{11} & s_{13} \\ s_{23} & s_{13} & s_{33} \end{bmatrix}$$

The permutation step is essential for hardware implementation because it prevents division by zero conditions during LDL^T factorization and significantly improves numerical stability. In FPGA-based designs, where predictable and safe arithmetic is crucial, ensuring that the pivot structure is valid is a necessary preprocessing step before performing any factorization or eigenvalue computation.

The LDL^T is normally introduced as producing a diagonal matrix D for positive definite matrices. However, this is not the case for indefinite or singular matrices such as the ellipse constraint matrix. After permutation, the upper-left portion of C contains a pattern in which both the (1,1) and (3,3) diagonal entries are zero but the off-diagonal entries (1,3) and (3,1) are non-zero. The LDL^T factorization invokes Bunch-Kaufman pivoting, which produces a 2x2 pivot block in D, rather than two independent 1x1 pivots. As a result, the matrix D becomes block diagonal, consisting of one 1x1 block from the -1 pivot, and one 2x2 block arising from the off-diagonal coupling between the quadratic terms a and c. The remaining 3x3 zero block corresponds to the unconstrained linear and constant parameters (d,e,f). This structure is crucial, since the 2x2 blocks must be inverted as a full matrix, while the zero block cannot be inverted at all. These properites directly how the generalized eigen-problem is reduced. Since the lower 3x3 block of C is zero, the corresponding directions satisfy Ca = 0, which produces infinite eigenvalues in the generalized eigenproblem $Sa = \lambda Ca$. The LDL^T reduction isolates these directions so the finite eigenvalues are solved for.
Furthermore, the right-hand side of equation (7) is then simplified to remove the triangular matrices L and LT. The inverse of L is then multiplied by both sides of the equation to help simplify the equation into the standard eigenvalue problem

$$L^{-1}Sa = \lambda(L^{-1}LDL^T)a \tag{10}$$

which can be reduced to:

$$L^{-1}Sa = \lambda DL^T a \tag{11}$$

Then a new variable (12) is substituted to recover the eigenvectors of variable a of the original problem.

$$z = L^T a => (L^{-T})z = a => a = L^{-T}z \tag{12}$$

Equation (11) is then taken and a to z is taken to become:

$$L^{-1}S(L^{-T}z) = \lambda DL^T(L^{-T}z) \tag{13}$$

This is reduced to:

$$L^{-1}SL^{-T}z = \lambda Dz \tag{14}$$

(11) is then normalized, which is done by multiplying both sides of the equation by the inverse of D, so that we get left with equation (15)

$$\lambda z \tag{15}$$

(lambda are the eigenvalues of the standard eigenvalue problem, and z is the eigenvectors, which will correspond to the ellipse solution).

$$(D^{-1}L^{-1})SL^{-T}z = \lambda(D^{-1}D)z \tag{16}$$

which is reduced to:

$$D^{-1}L^{-1}SL^{-T}z = \lambda z \tag{17}$$

This can also be written as

$$Az = \lambda z \tag{18}$$

The matrix A is then symmetric on the constrained subspace, which is essential because the Jacobi eigenvalue algorithm requires the matrix to be symmetric. The symmetry arises from two sources. The scatter matrix $S = D^T * D$ is inherently symmetric and positive semi-definite. Then, the similairity transformation used in the reduction ($L^{-1}SL^{-T}$) preserves the symmetry. The inverse of D is premultiplied to maintain the

symmetry when applied only to the invertible block, since the block is symmetric. Thus, although the original generalized eigen-problem involves an indefinite and singular matrix C, the reduced A is well defined and symmetric, enabling robust computation of eigen-pairs.

While mathematically correct, the LDL^T approach proved complex for hardware implementation due to the permutation logic, multiple matrix inversions, and high resource cost in Vitis HLS.

A. Final Approach: Schur Complement Reduction and $\mathbf{S}_{tilde}$

To overcome these limitations, the Schur complement reduction was adopted. The 6×6 scatter matrix S is partitioned into blocks S_{pp}, S_{pq}, and S_{qq}. The reduced 3×3 matrix is formed as

$$\mathbf{S}_{\text{tilde}} = S_{pp} - S_{pq}\, S_{qq}^{-1}\, S_{qp}.$$

A fixed transformation matrix D (derived from the ellipse constraint $4ac - b^2 = 1$) is then applied:

$$A = D^{-1}\mathbf{S}_{\text{tilde}},$$

where

$$D^{-1} = \begin{bmatrix} 0 & 0.5 & 0 \\ 0.5 & 0 & 0 \\ 0 & 0 & -1 \end{bmatrix}.$$

This produces a standard symmetric 3×3 eigenproblem

$$A\mathbf{v} = \lambda\mathbf{v},$$

which is solved efficiently using the cyclic Jacobi method. The resulting approach is numerically stable, significantly smaller (3×3 instead of 6×6), and far more synthesizable in Vitis HLS.

The cyclic Jacobi algorithm iteratively applies Givens rotations to diagonalize A. For each iteration, the largest off-diagonal element A_{pq} is selected, and a rotation is applied to zero it while preserving symmetry. After convergence, the eigenvector corresponding to the smallest positive eigenvalue is scaled to enforce the constraint $4ac - b^2 = 1$, yielding the final ellipse coefficients $[a, b, c, d, e, f]$.

This Schur-complement-based method eliminates the need for LDL^T factorization entirely and is the core of the implemented Vitis HLS kernel.

Initially, the Jacobi method takes the largest off-diagonal of the symmetric matrix Apq. The two indices (p,q) identify the plane of rotation, which means that the Jacobi method applies a rotation angle in the plane (p,q), which makes Apq = 0.

$$(p, q) = \arg\max_{p<q} |A_{pq}| \tag{19}$$

$$\tan(2\theta) = \frac{2A_{pq}}{A_{qq} - A_{pp}} \tag{20}$$

$$\text{t} = \begin{cases} \frac{1}{\tau+\sqrt{1+\tau^2}}, & \tau \geq 0, \\ -\frac{1}{-\tau+\sqrt{1+\tau^2}}, & \tau < 0, \end{cases} \quad \text{where} \quad \tau = \frac{A_{qq}-A_{pp}}{2A_{pq}}.$$

For the rotation in the (p, q) plane, only the p-th and q-th rows and columns of A are modified. The updated diagonal and off-diagonal elements are computed as:

$$A'_{pp} = c^2 A_{pp} - 2scA_{pq} + s^2 A_{qq}, \tag{21}$$

$$A'_{qq} = s^2 A_{pp} + 2scA_{pq} + c^2 A_{qq}, \tag{22}$$

$$A'_{pq} = A'_{qp} = 0, \tag{23}$$

and for all $i \neq p, q$:

$$A'_{ip} = cA_{ip} - sA_{iq}, \tag{24}$$

$$A'_{iq} = sA_{ip} + cA_{iq}. \tag{25}$$

These updates maintain the symmetry of the matrix after each rotation.

IV. Experimental Setup

The proposed ellipse-fitting accelerator was evaluated using real shock and vibration data collected from an accelerometer. The primary dataset used is `Temp_21_output_time.csv`, which contains time-stamped acceleration samples. The sampling interval Δt was extracted directly from the time column to compute the frequency spectrum via FFT and to determine the optimal time delay and window size.

Key parameters for the sliding-window approach were derived automatically from the spectrum:

- Maximum frequency $F_{\max}$ and minimum frequency $F_{\min}$
- Time delay = $0.25/F_{\max}$ (converted to number of samples)
- Window duration = $1/F_{\min}$ (converted to number of samples)

A fixed window size of 100 samples was used to match the hardware implementation (`WINDOW_SIZE = 100`).

The design was synthesized using **Vitis HLS 2024.1** targeting a Xilinx Virtex UltraScale+ device (`xcvu9p-flga2104-2-i`) with a target clock period of 5 ns (200 MHz). The top-level function `scatter_matrix` processes a streaming `int16_t` input and outputs the six ellipse coefficients $[a, b, c, d, e, f]$ via an AXI-Lite interface. Numerical correctness was validated by comparing the HLS output against the MATLAB golden reference function `fit_ellipse_hls_golden`.

Performance metrics collected include latency (cycles), initiation interval (II), and resource utilization (DSP, FF, LUT). All experiments were performed on the default solution with full array partitioning and pipeline directives.

V. Results

C synthesis results for the `scatter_matrix` kernel are summarized in Table I.

Figure 2 shows the acceleration time series (top) with three representative phase-space orbits (bottom insets). The blue shaded regions mark the windows before, during, and after the impact event. The orbits remain clean and elliptical before and after the impact, but become highly distorted during the shock.

The design successfully processes streaming accelerometer data and produces ellipse parameters that match the MATLAB golden reference within floating-point tolerance.

TABLE I: Performance and Resource Estimates after C Synthesis (Vitis HLS 2024.1)

Module / Function	Latency (cycles)	Latency (μs)	II	DSP
scatter_matrix (top level)	870	4.35	3	1432
jacobi_eigen_3x3	695	3.475	1	1247
atan2_cordic_float_s	38	0.19		
sin_or_cos_float_s	9	0.045	1	—

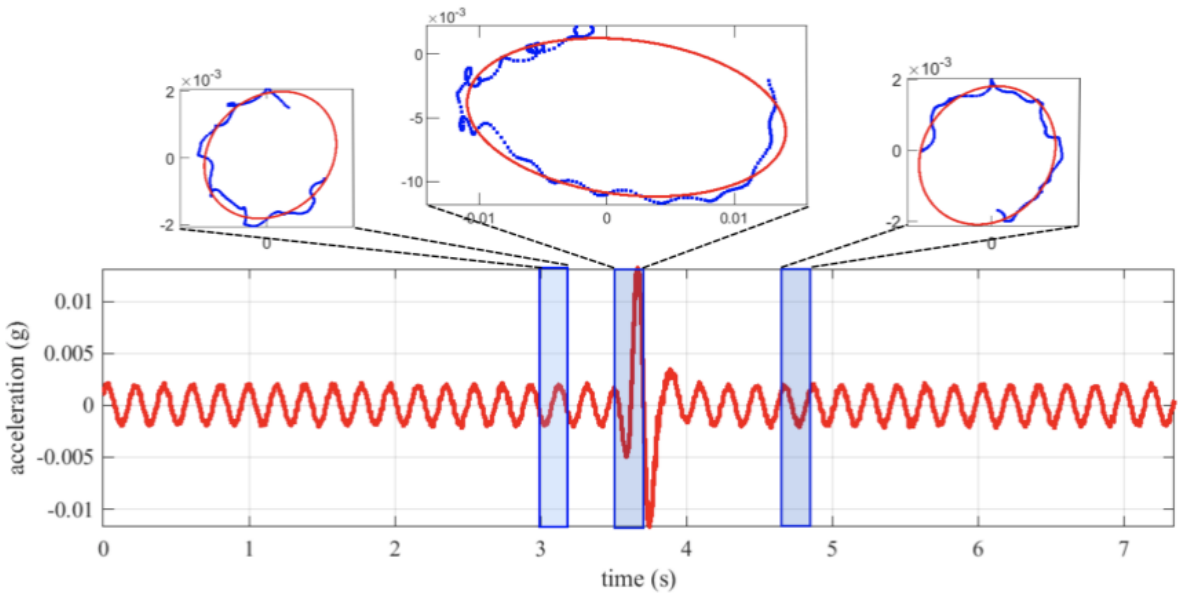

Fig. 2: Acceleration time series (top) showing normal vibration and the impact event at approximately $t = 4$ s. The three insets show the phase-space orbits (acceleration vs. delayed acceleration) extracted by the ellipse fitting algorithm. The orbits remain clean and elliptical before and after the impact, while the orbit becomes highly distorted during the shock.

VI. Discussions and Conclusions

The Schur complement reduction combined with the fixed transformation matrix D^{-1} proved to be a highly effective approach for hardware implementation. By reducing the original 6×6 generalized eigenproblem to a compact 3×3 standard eigenproblem, we eliminated the need for complex LDL^T factorization, permutation logic, and multiple matrix inversions. This resulted in a much more synthesizable design with significantly lower resource pressure and improved predictability.

The current initiation interval of 3 is acceptable for real-time vibration monitoring, but further optimizations (deeper loop unrolling, dataflow partitioning, or reduced-precision fixed-point arithmetic) could bring the II down to 1. The design successfully processes streaming accelerometer data and produces ellipse parameters that match the MATLAB golden reference within floating-point tolerance.

In conclusion, this work demonstrates that a Schur-complement-based ellipse fitting accelerator can be efficiently realized on an FPGA using Vitis HLS. The final implementation is numerically stable, hardware-friendly, and suitable for real-time shock/vibration anomaly detection. Future work will focus on achieving II=1, integrating geometric parameter extraction (center, axes, angle), and deploying the kernel on actual hardware for live anomaly detection.